HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

LÝ THUYẾT MẬT MÃ

TS. TẠ KIM HUỆ
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ONE LOVE. ONE FUTURE.

Recap: message integrity

Có 4 cấu trúc của MAC

randomized MAC (e.g. 802.11i)

ECBC-MAC, CMAC: commonly used with AES

PRFs

NMAC : basis of HMAC (this segment)

PMAC: a parallel MAC

Carter-Wegman MAC: built from a fast one-time MAC

This module: MACs from collision resistance (kháng va chạm).

Kháng va chạm(Collision Resistance)

Let H: M $\rightarrow$T be a hash function (|M| >> |T|)

Một va chạm với hàm H là tồn tại cặp m_0 , $m_1 \in M$ sao cho:

$$H(m_0) = H(m_1) \quad \text{and} \quad m_0 \neq m_1$$

Hàm H là kháng va chạm (**collision resistant) nếu :** if for all (explicit) "eff" algs. A:

$Adv_{CR}[A,H]$ = Pr[A outputs collision for H] is "neg".

Example: SHA – 256 (outputs 256 bits), SHA – 512 (outputs 512 bits)

MACs from Collision Resistance

Let I = (S,V) be a MAC for short messages over (K,M,T) (e.g. AES)

Let H: $M^{big} \rightarrow M$

Def: $I^{big} = (S^{big}, V^{big})$ over (K, M^{big}, T) as:

$$S^{big}(k,m) = S(k,H(m)) \quad ; \quad V^{big}(k,m,t) = V(k,H(m),t)$$

<u>Định lý (Thm)</u>: Nếu I là một MAC bảo mật và hàm H kháng va chạm thì I^{big} là một MAC bảo mật.

Example: $S(k,m) = AES_{2\text{-block-cbc}}(k, \text{SHA-256}(m))$ is a secure MAC.

MACs from Collision Resistance

Kháng va chạm là cần thiết cho bảo mật:

Giả thiết kẻ địch (adversary) có thể tìm $m_0 \neq m_1$ sao cho $H(m_0) = H(m_1)$. Do đó: S^{big} là không bảo mật với tấn công 1-chosen msg attack

step 1: adversary asks for $t \leftarrow S(k, m_0)$

step 2: output (m_1, t) as forgery (giả mạo)

$$S^{big}(k, m) = S(k, H(m)) \quad ; \quad V^{big}(k, m, t) = V(k, H(m), t)$$

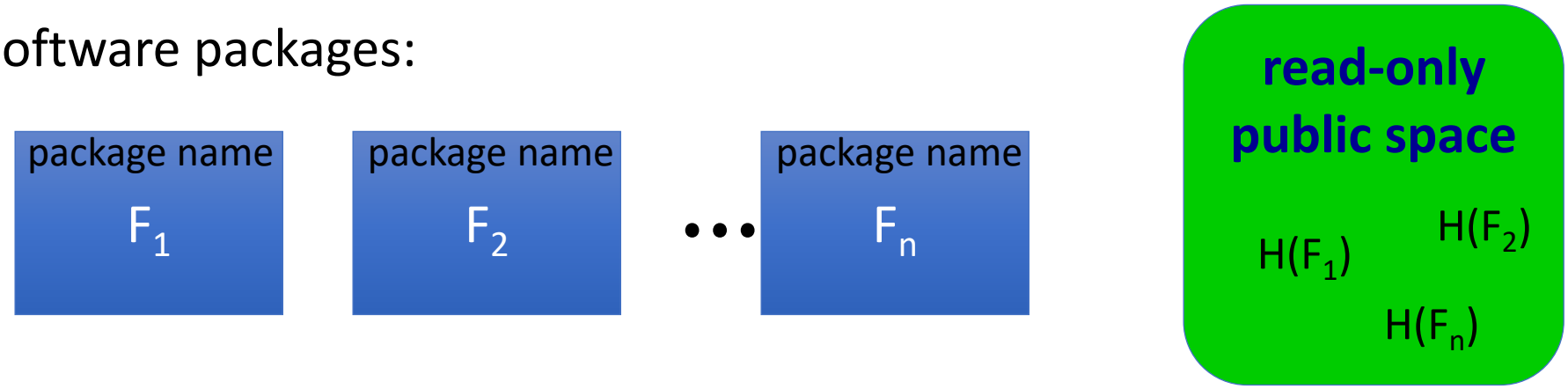
Protecting file integrity using C.R. hash

Khi người dùng tải các gói tin, có thể xác nhận được nội dung có hợp lệ hay không

H collision resistant ⇒ attacker không thể thay đổi được gói tin mà không bị phát hiện (cannot modify package without detection)

no key needed (public verifiability), but requires read-only space

Software packages:



Generic attack on C.R. functions

Let $H: M \rightarrow \{0,1\}^n$ là một hàm băm ($|M| >> 2^n$)

Generic alg. tìm va chạm trong thời gian **(in time)** **$O(2^{n/2})$** hashes

Algorithm:

1. Chọn **$2^{n/2}$** bản tin ngẫu nhiên trong M: $m_1, \ldots, m_{2^{n/2}}$ (distinct w.h.p)
2. For i = 1, …, $2^{n/2}$ tính $t_i = H(m_i)$ $\in\{0,1\}^n$
3. Tìm kiếm 1 va chạm ($t_i = t_j$). Nếu không tìm thấy, lặp lại bước 1

How well will this work?

Nguyên lý ngày sinh nhật (The birthday

Cho $r_1, \ldots, r_n \in \{1,\ldots,B\}$ là có số nguyên xác định phân bố độc lập (indep. identically distributed integers).

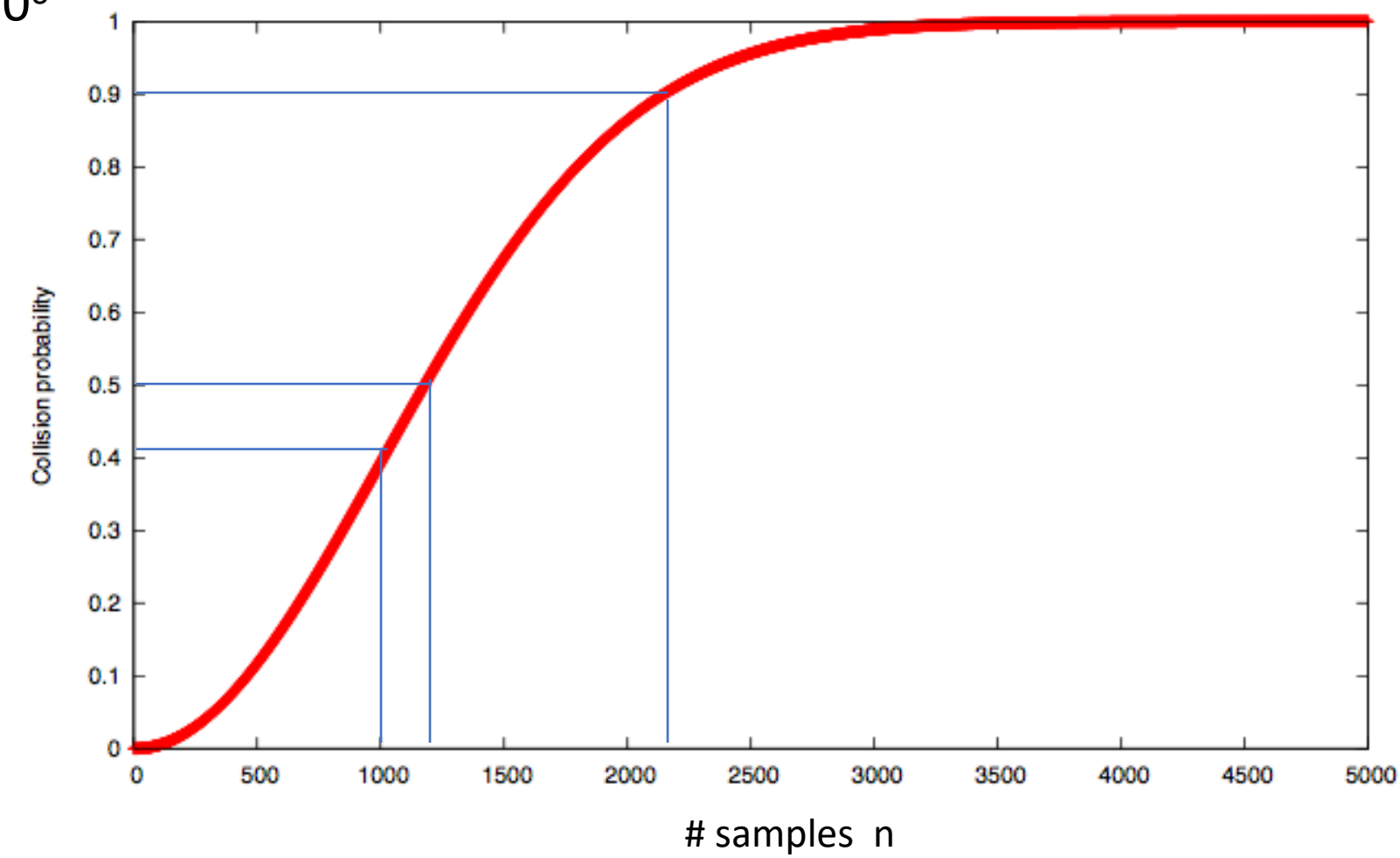
Thm: khi $n = 1.2 \times B^{1/2}$ thì $\Pr[\exists i \neq j: r_i = r_j] \geq ½$

Proof: (for uniform indep. $r_1, \ldots, r_n$)

$$\Pr[\exists\, i \neq j: r_i = r_j] = 1 - \Pr[\exists\, i \neq j: r_i \neq r_j] = 1 - \left(\frac{B-1}{B}\right)\left(\frac{B-2}{B}\right)\ldots\left(\frac{B-n+1}{B}\right) = 1 - \prod_{i=1}^{n-1}\left(1 - \frac{i}{B}\right) \geq 1 - \prod_{i=1}^{n-1}\left(e^{-i/B}\right) = 1 - e^{-\frac{i}{B\sum_{i=1}^{n-1} i}} \geq 1 - e^{-\frac{n^2}{2B}} \geq 0.53 > 1/2$$

(where, $1 - x \leq e^{-x}$, $\frac{n^2}{2B} = 0.72$)

$B=10^6$


Collision probability
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
500
1000
1500
2000
2500
3000
3500
4000
4500
5000
samples n

Generic attack

$H: M \rightarrow \{0,1\}^n$. Collision finding algorithm:

1. Choose $\mathbf{2^{n/2}}$ random elements in M: $m_1, \ldots, m_{2^{n/2}}$
2. For i = 1, …, $2^{n/2}$ compute $t_i = H(m_i) \in \{0,1\}^n$
3. Look for a collision ($t_i = t_j$). If not found, got back to step 1.

Expected number of iteration ≈ 2

Running time: $\mathbf{O(2^{n/2})}$ (space $O(2^{n/2})$)

Sample C.R. hash functions: Crypto++ 5.6.0 [Wei Dai]

AMD Opteron, 2.2 GHz (Linux)

	function	digest size (bits)	Speed (MB/sec)	generic attack time
	SHA-1	160	153	2^{80}
NIST standards	SHA-256	256	111	2^{128}
NIST standards	SHA-512	512	99	2^{256}
	Whirlpool	512	57	2^{256}

* best known collision finder for SHA-1 requires 2^{51} hash evaluations

Quantum Collision Finder

	Classical algorithms	**Quantum algorithms**
Block cipher **E: K × X ⟶ X** exhaustive search	$O(\|K\|)$	$O(\|K\|^{1/2})$
Hash function **H: M ⟶ T** collision finder	$O(\|T\|^{1/2})$	$O(\|T\|^{1/3})$

Collision resistance: review

Let $H: M \rightarrow T$ be a hash function ($|M| >> |T|$)

A **collision** for H is a pair $m_0, m_1 \in M$ such that:

$$H(m_0) = H(m_1) \quad \text{and} \quad m_0 \neq m_1$$

Goal: tạo ra hàm băm kháng va chạm (collision resistant (C.R.) hash functions)

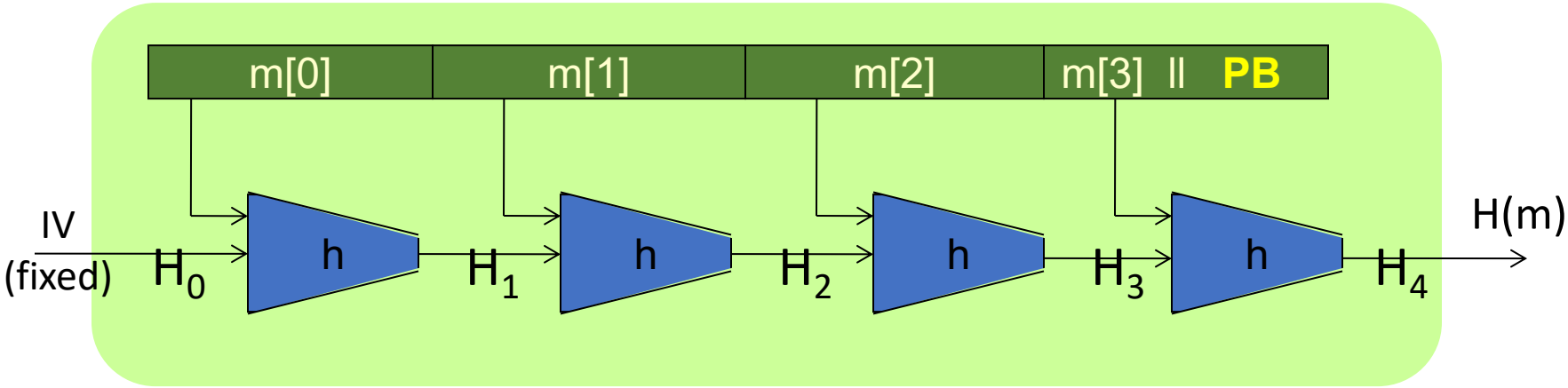
Step 1: given C.R. function for **short** messages, construct C.R. function for **long** messages

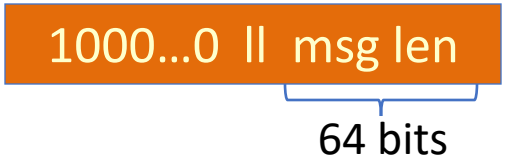
The Merkle-Damgard iterated construction

Given **$h: T \times X \longrightarrow T$** (compression function)

we obtain **$H: X^{\leq L} \longrightarrow T$** . H_i - chaining variables

PB: padding block



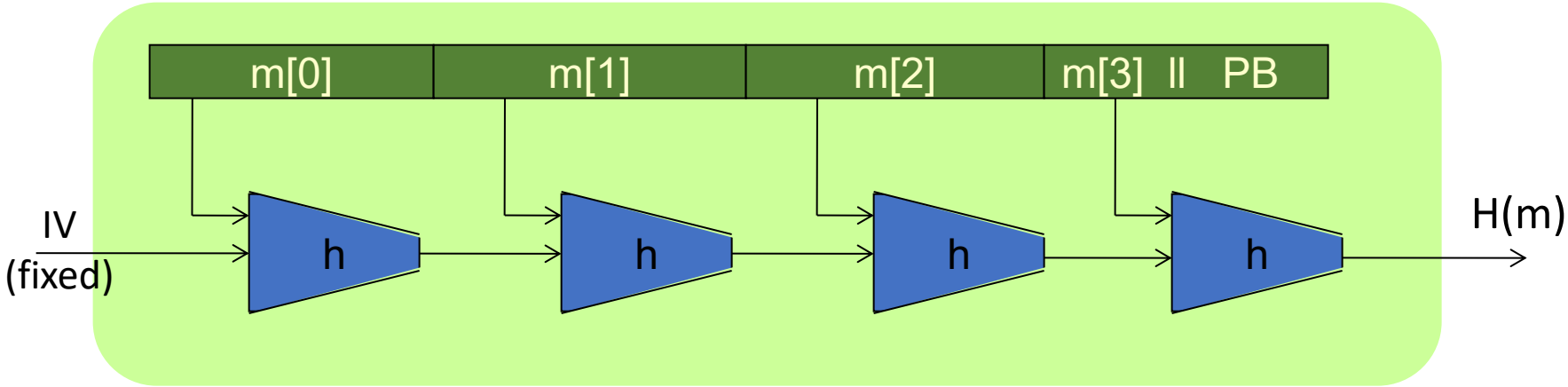


If no space for PB add another block

The Merkle-Damgard iterated construction

Thm: h kháng va chạm ⇒ H kháng va chạm

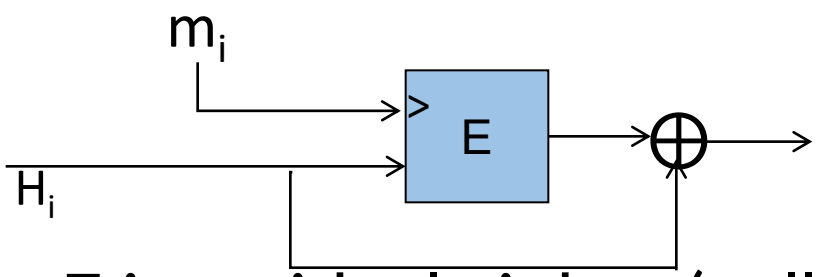
Goal: cấu trúc hàm nén **h: T × X ⟶ T**



Compr. func. from a block cipher

E: K× $\{0,1\}^n \longrightarrow \{0,1\}^n$ là hàm mật mã khối .

The **Davies-Meyer** compression function: **h(H, m) = E(m, H)⊕H**



<u>Thm</u>: Suppose E is an ideal cipher (collection of |K| random perms.).

Finding a collision **h(H,m)=h(H',m')** takes **$O(2^{n/2})$** evaluations of (E,D).

Best possible !!

Other block cipher constructions

Miyaguchi-Preneel: $h(H, m) = E(m, H) \oplus H \oplus m$
(Whirlpool)

$$h(H, m) = E(H \oplus m, m) \oplus m$$

total of 12 variants like this

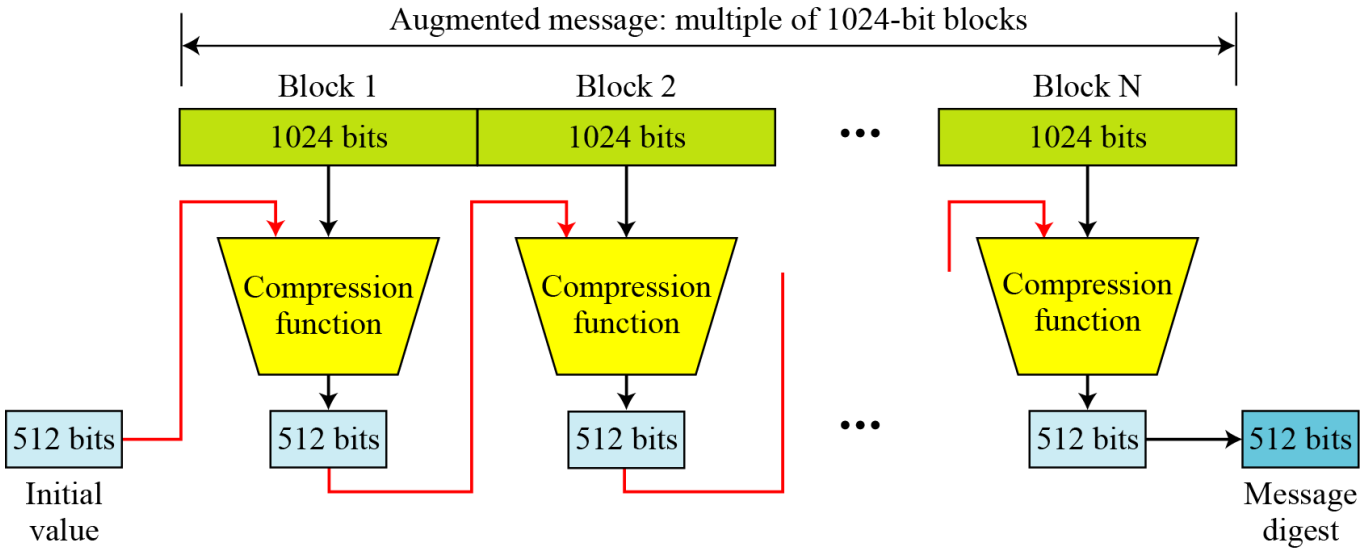
Let $E: \{0,1\}^n \times \{0,1\}^n \longrightarrow \{0,1\}^n$ for simplicity

Other natural variants are insecure:

$$h(H, m) = E(m, H) \oplus m \qquad \text{(HW)}$$

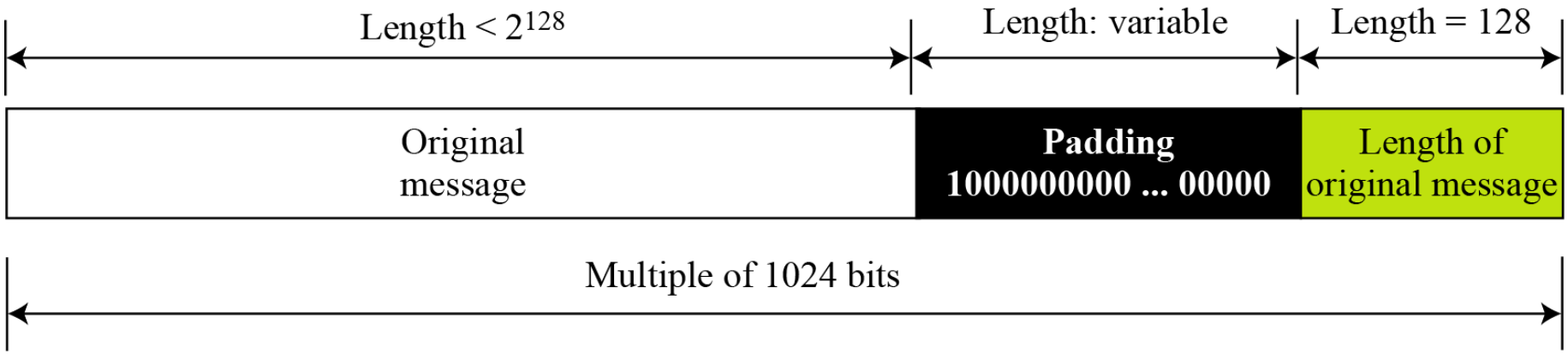
Case study SHA-512

- SHA-512 là phiên bản của SHA với bản mã băm 512 bit, dựa trên lược đồ Merkle-Damgard
- SHA-512 tạo bản băm 512-bit với đầu vào nhỏ hơn 2^{128} (hiệu năng xử lý tối ưu)



Trường Padding và length

Ví dụ: Số bit đệm Padding là bao nhiêu nếu độ dài của bản tin gốc là 2590 bit?



(M+P+128) ≡ 0 mod 1024 → P (padding) ≡ (-M-128) mod 1024

$$|P| = (-2590 - 128) \bmod 1024 = -2718 \bmod 1024 = 354$$

Phần đệm bao gồm một số 1 theo sau là 353 số 0.

Chúng ta có cần đệm không nếu độ dài của tin nhắn gốc đã là bội số của 1024 bit?

Có, bởi vì chúng ta cần thêm trường độ dài. Vì vậy, cần đệm để tạo khối mới là bội số của 1024 bits.

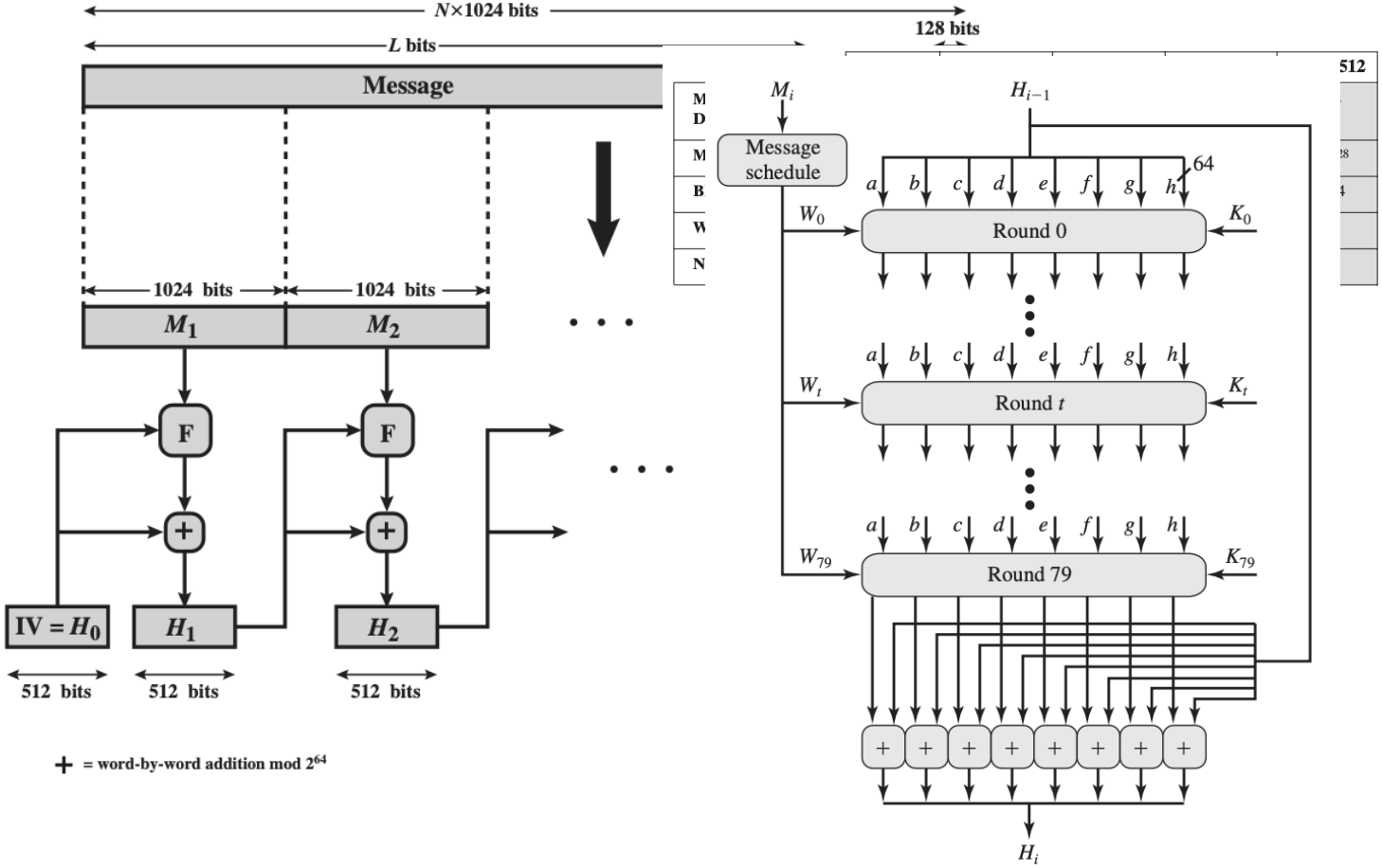
Trường Padding và length

Số bit đệm tối thiểu và tối đa có thể được thêm vào bản tin là bao nhiêu?

Độ dài tối đa của vùng đệm là 1023 và điều đó xảy ra khi (- | M | −128) = 1023 mod 1024. Điều này có nghĩa là độ dài của thông điệp gốc là | M | = (−128 −1023) mod 1024 hoặc độ dài là | M | = 897 mod 1024. Trong trường hợp này, chúng ta không thể chỉ thêm trường độ dài vì độ dài của khối cuối cùng vượt quá một bit nhiều hơn 1024. Vì vậy, chúng ta cần thêm 897 bit để hoàn thành khối này và tạo khối thứ hai gồm 896 bit. Bây giờ chiều dài có thể được thêm vào để làm cho khối này hoàn chỉnh.

Độ dài tối thiểu của padding là 0 và nó xảy ra khi (−M - 128) mod 1024 bằng 0. Điều này có nghĩa là | M | = −128 mod 1024 = 896 mod 1024 bit. Nói cách khác, khối cuối cùng trong thông điệp gốc là 896 bit. Chúng ta thêm trường độ dài 128 bit để làm cho khối hoàn chỉnh.

SHA - 512

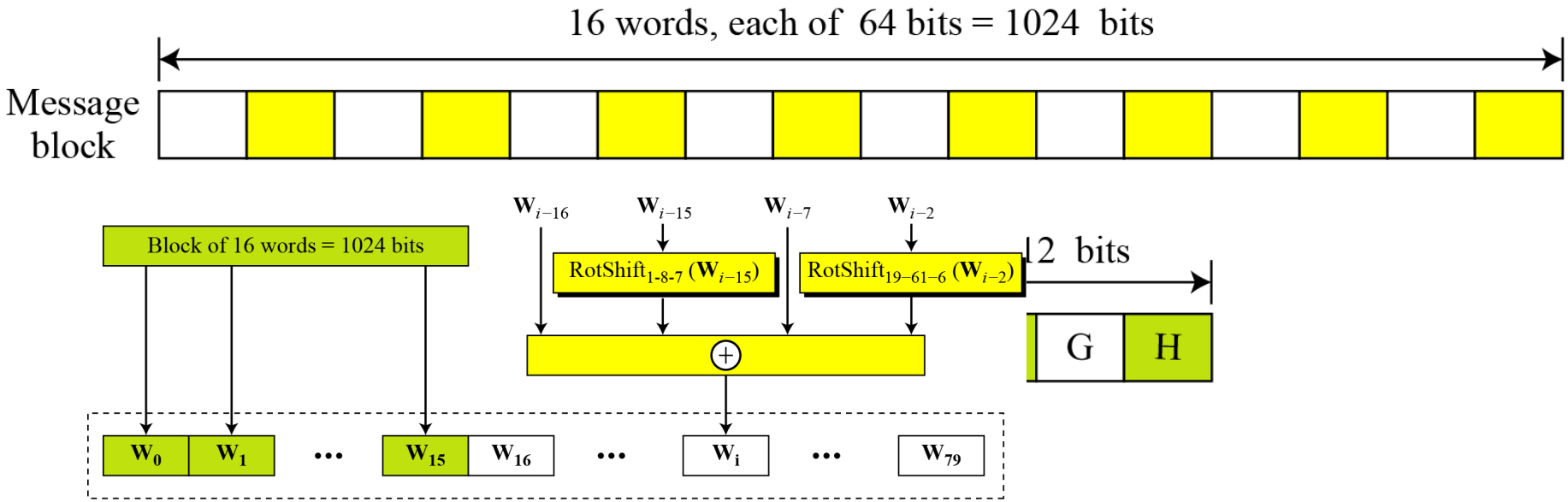

N×1024 bits
L bits
Message
1024 bits
1024 bits
M1
M2
F
F
IV = H0
H1
H2
512 bits
512 bits
512 bits
+ = word-by-word addition mod 2^64
128 bits
512
Mi
Hi−1
Message schedule
64
a b c d e f g h
W0
Round 0
K0
Wt
Round t
Kt
W79
Round 79
K79
Hi

Word and Word Expansion

Mỗi từ trong phạm vi W16 đến W79 được tạo từ bốn từ đã tạo trước đó. W60 được làm như sau

A message block and the message digest as words

Cho biết W60 được tạo ra như thế nào



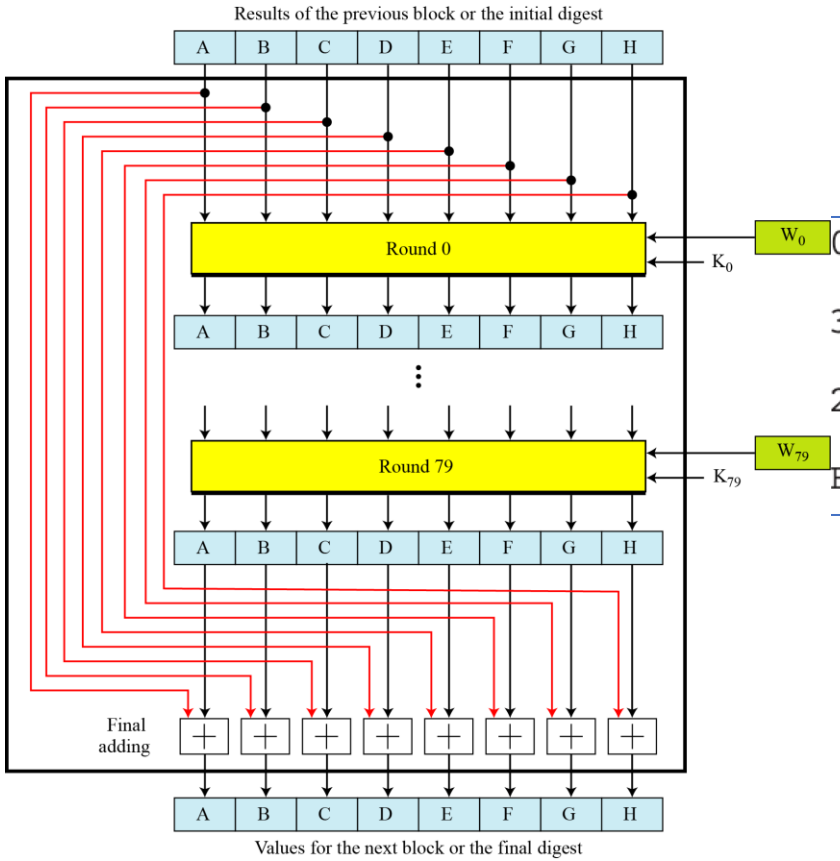
$RotShift_{1-m-n}(x)$: $RotR_l(x) \oplus RotR_m(x) \oplus ShL_n(x)$

$RotR_i(x)$: Right-rotation of the argument x by i bits

$ShL_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

$$W_{60} = W_{44} \oplus RotShift_{1-8-7}(W_{45}) \oplus W_{53} \oplus RotShift_{19-61-6}(W_{58})$$

Message Digest Initialization




```
08   e = 510E527FADE682D1

3B   f = 9B05688C2B3E6C1F

2B   g = 1F83D9ABFB41BD6B

F1   h = 5BE0CD19137E2179
```

Inside Round function

Majority Function

$$(A_j \text{ AND } B_j) \oplus (B_j \text{ AND } C_j) \oplus (C_j \text{ AND } A_j)$$

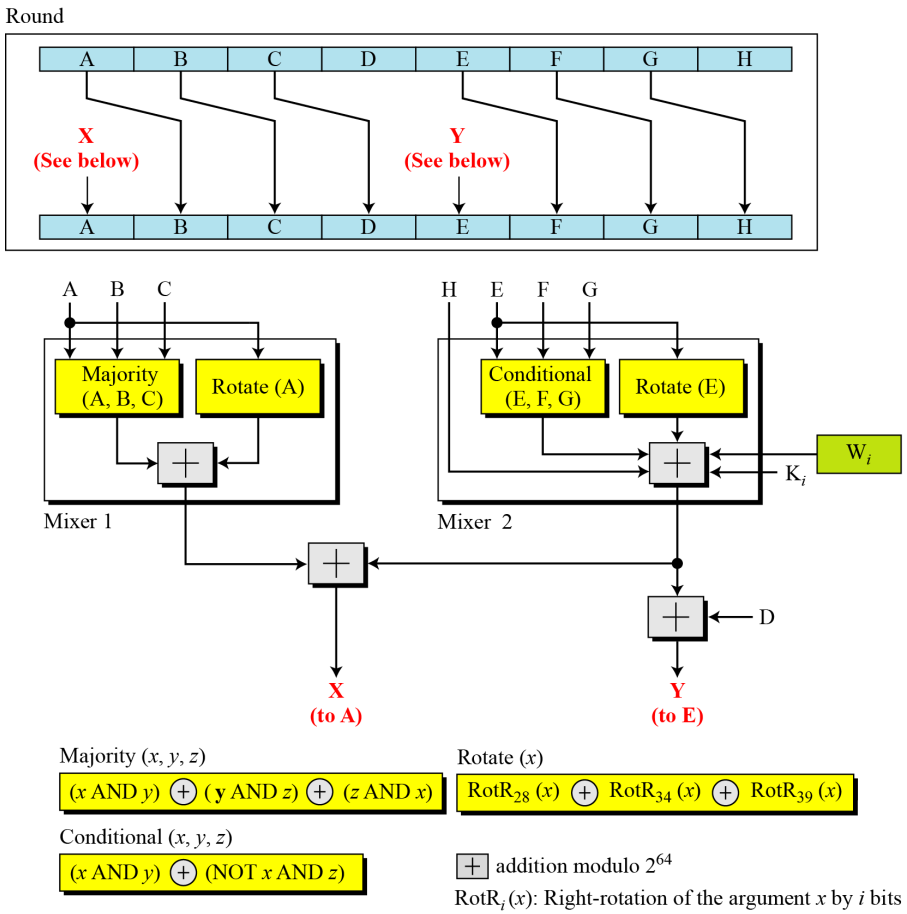
Conditional Function

$$(E_j \text{ AND } F_j) \oplus (\text{NOT } E_j \text{ AND } G_j)$$

Rotate Functions

$$\text{Rotate (A): } \text{RotR}_{28}(A) \oplus \text{RotR}_{34}(A) \oplus \text{RotR}_{29}(A)$$

$$\text{Rotate (E): } \text{RotR}_{28}(E) \oplus \text{RotR}_{34}(E) \oplus \text{RotR}_{29}(E)$$



Ví dụ về hàm Majority

- Áp dụng hàm Majority cho các giá trị đệm A, B, and C. Nếu các số hexa phía trái nhất của các giá trị này tương ứng là là 0x7, 0xA, and 0xE, thì giá trị số hexa trái nhất của kết quả hàm Majority là bao nhiêu?
- Solution: The digits in binary are 0111, 1010, and 1110.
  - The first bits are 0, 1, and 1. The majority is 1.
  - The second bits are 1, 0, and 1. The majority is 1.
  - The third bits are 1, 1, and 1. The majority is 1.
  - The fourth bits are 1, 0, and 0. The majority is 0.
- The result is 1110, or 0xE in hexadecimal.

Ví dụ về hàm Conditional

- Áp dụng hàm Conditional với các giá trị đệm E, F, and G. Nếu số hexa trái nhất của các giá trị đệm này lần lượt là 0x9, 0xA, and 0xF, số trái nhất trong kết quả đầu ra là bao nhiêu?
- Solution: The digits in binary are 1001, 1010, and 1111.
  - The first bits are 1, 1, and 1. The result is F_1, which is 1.
  - The second bits are 0, 0, and 1. The result is G_2, which is 1.
  - The third bits are 0, 1, and 1. The result is G_3, which is 1.
  - The fourth bits are 1, 0, and 1. The result is F_4, which is 0.
- The result is 1110, or 0xE in hexadecimal.

Constants

Table 12.3 *Eighty constants used for eighty rounds in SHA-512*

428A2F98D728AE22	7137449123EF65CD	B5C0FBCFEC4D3B2F	E9B5DBA58189DBBC
3956C25BF348B538	59F111F1B605D019	923F82A4AF194F9B	AB1C5ED5DA6D8118
D807AA98A3030242	12835B0145706FBE	243185BE4EE4B28C	550C7DC3D5FFB4E2
72BE5D74F27B896F	80DEB1FE3B1696B1	9BDC06A725C71235	C19BF174CF692694
E49B69C19EF14AD2	EFBE4786384F25E3	0FC19DC68B8CD5B5	240CA1CC77AC9C65
2DE92C6F592B0275	4A7484AA6EA6E483	5CB0A9DCBD41FBD4	76F988DA831153B5
983E5152EE66DFAB	A831C66D2DB43210	B00327C898FB213F	BF597FC7BEEF0EE4
C6E00BF33DA88FC2	D5A79147930AA725	06CA6351E003826F	142929670A0E6E70
27B70A8546D22FFC	2E1B21385C26C926	4D2C6DFC5AC42AED	53380D139D95B3DF
650A73548BAF63DE	766A0ABB3C77B2A8	81C2C92E47EDAEE6	92722C851482353B
A2BFE8A14CF10364	A81A664BBC423001	C24B8B70D0F89791	C76C51A30654BE30
D192E819D6EF5218	D69906245565A910	F40E35855771202A	106AA07032BBD1B8
19A4C116B8D2D0C8	1E376C085141AB53	2748774CDF8EEB99	34B0BCB5E19B48A8
391C0CB3C5C95A63	4ED8AA4AE3418ACB	5B9CCA4F7763E373	682E6FF3D6B2B8A3
748F82EE5DEFB2FC	78A5636F43172F60	84C87814A1F0AB72	8CC702081A6439EC
90BEFFFA23631E28	A4506CEBDE82BDE9	BEF9A3F7B2C67915	C67178F2E372532B
CA273ECEEA26619C	D186B8C721C0C207	EADA7DD6CDE0EB1E	F57D4F7FEE6ED178
06F067AA72176FBA	0A637DC5A2C898A6	113F9804BEF90DAE	1B710B35131C471B
28DB77F523047D84	32CAAB7B40C72493	3C9EBE0A15C9BEBC	431D67C49C100D4C
4CC5D4BECB3E42B6	4597F299CFC657E2	5FCB6FAB3AD6FAEC	6C44198C4A475817

Ví dụ tạo hằng số

- Có 80 hằng số, K_0 to K_{79}, mỗi số có 64 bits. Các số này được xác định từ 80 số nguyên tố đầu tiên (2,3,5 …409). Ví dụ, số nguyên tố thứ 80 là 409, lấy căn bậc 3 (cubic root) của 409 là $(409)^{1/3}$ = 7.42291412044…
- Chuyển đổi dưới dạng biểu diễn nhị phân 64 bit ta có

$$(111.0110\ 1100\ 0100\ 0100\ \ldots\ 0111)_2 \rightarrow (7.6C44198C4A475817)_{16}$$

The fraction part: $(6C44198C4A475817)_{16}$

Bài tập SHA 512

- We include here an example based on one in FIPS 180. We wish to hash a one-block message consisting of three ASCII characters: “abc”, which is equivalent to the following 24-bit binary string: 01100001 01100010 01100011
- The padding consists of 896 - 24 = 872bits, consisting of a “1” bit followed by 871 “0” bits.

```
6162638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000018
```

- Tính các giá trị đầu ra của hàm vòng đầu tiên và thứ 2

Provable compression functions

Chọn một ngẫu nhiên một số nguyên tố p biểu diễn bởi 2000 bit và cặp số ngẫu nhiên $1 \le u, v \le p$.

Cho $m,h \in \{0,\ldots,p-1\}$ định nghĩa **$h(H,m) = u^H \cdot v^m \pmod p$**

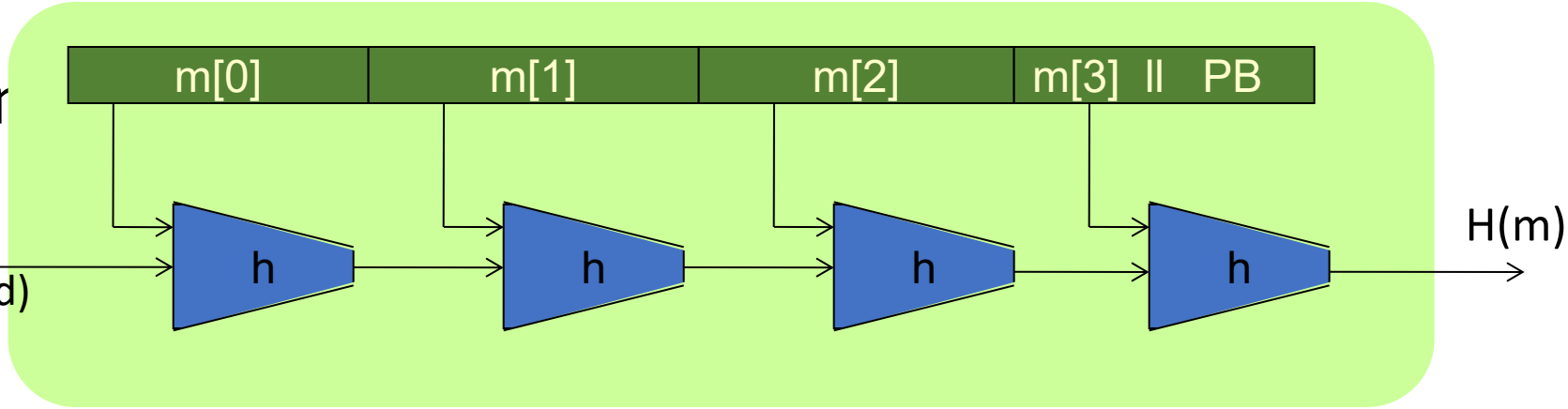
<u>Fact:</u> tìm va chạm của hàm h (.,.) có độ khó như bài toán solving "discrete-log" modulo p.

Problem: slow.

The Merkle-Damgard iterated construction

Thm: h collision resistant $\Rightarrow$ H collision resistant

Car



MAC from a Merkle-Damgard Hash Function

$H: X^{\leq L} \longrightarrow T$ a C.R. Merkle-Damgard Hash Function

<u>Attempt #1</u>: $S(k, m) = H(k \,\|\, m)$

This MAC is insecure because:

Given $H(k \,\|\, m)$ can compute $H(w \,\|\, k \,\|\, m \,\|\, PB)$ for any w.

Given $H(k \,\|\, m)$ can compute $H(k \,\|\, m \,\|\, w)$ for any w.

Given $H(k \,\|\, m)$ can compute $H(k \,\|\, m \,\|\, PB \,\|\, w)$ for any w.

Anyone can compute $H(k \,\|\, m)$ for any m.

Standardized method: HMAC (Hash-

Most widely used MAC on the Internet.

H: hash function.

example: SHA-256 ; output is 256 bits

Building a MAC out of a hash function:

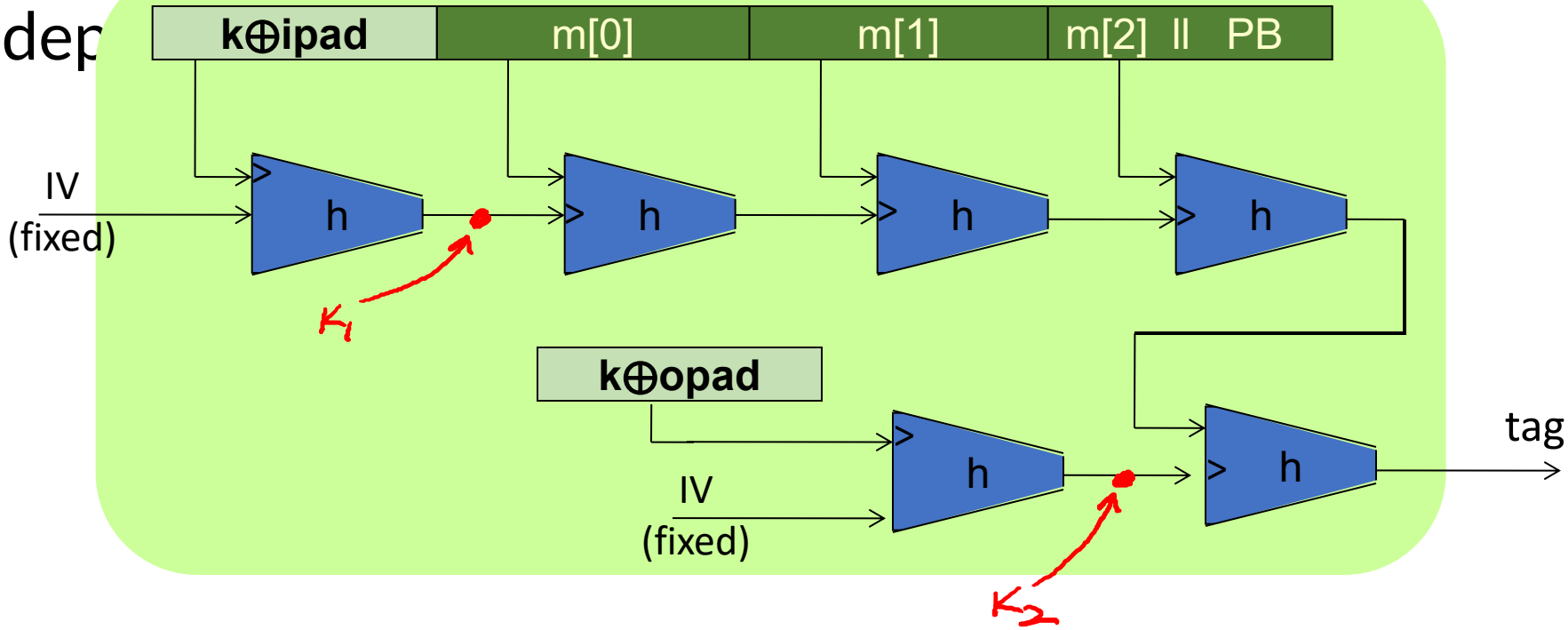
HMAC: $S(k, m) = H(k \oplus opad \,||\, \mathbf{H(k \oplus ipad \,||\, m)})$

HMAC in pictures

Similar to the NMAC PRF.

main difference: the two keys k_1, k_2 are

dep



HMAC properties

Built from a black-box implementation of SHA-256.

HMAC is assumed to be a secure PRF

- Can be proven under certain PRF assumptions about h(.,.)
- Security bounds similar to NMAC
  - Need $q^2/|T|$ to be negligible ($q << |T|^{1/2}$)

In TLS: must support HMAC-SHA1-96

Warning: verification timing attacks [L'09]

Example: Keyczar crypto library (Python) [simplified]

```
def Verify(key, msg, sig_bytes):
    return HMAC(key, msg) == sig_bytes
```

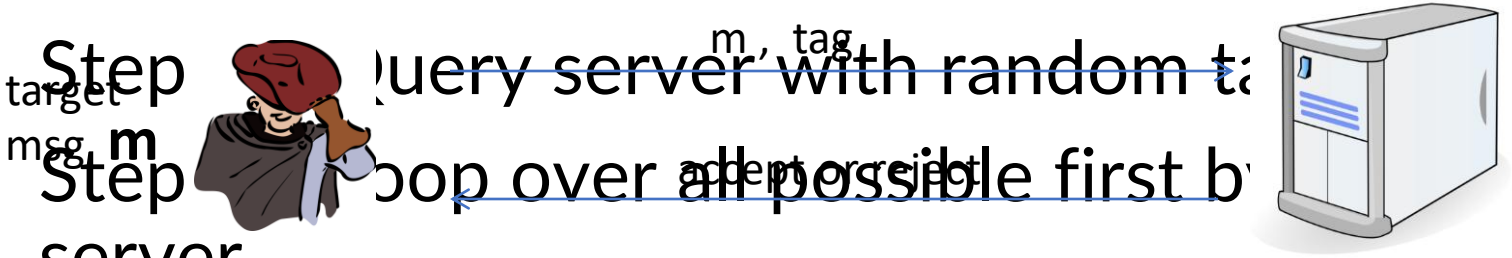
The problem: '==' implemented as a byte-by-byte comparison

- Comparator returns false when first inequality found

Warning: verification timing attacks [L'09]

Timing attack: to compute tag for target message m do:

Step 1: query server with random tag k



Step 2: loop over all possible first bytes query server.

stop when verification takes a little longer than in step 1

Step 3: repeat for all tag bytes until valid tag found

3	53	*	*	*	*

Defense #1

Make string comparator always take same time (Python) :

```
return false if  sig_bytes  has wrong length
result = 0
for x, y in zip( HMAC(key,msg) , sig_bytes):
    result |= ord(x) ^ ord(y)
return result == 0
```

Can be difficult to ensure due to optimizing compiler.

Defense #2

Make string comparator always take same time (Python) :

```
def Verify(key, msg, sig_bytes):
    mac = HMAC(key, msg)
    return HMAC(key, mac) == HMAC(key, sig_bytes)
```

Attacker doesn't know values being compared

MACs based on Hash function - HMAC (RFC 2104)

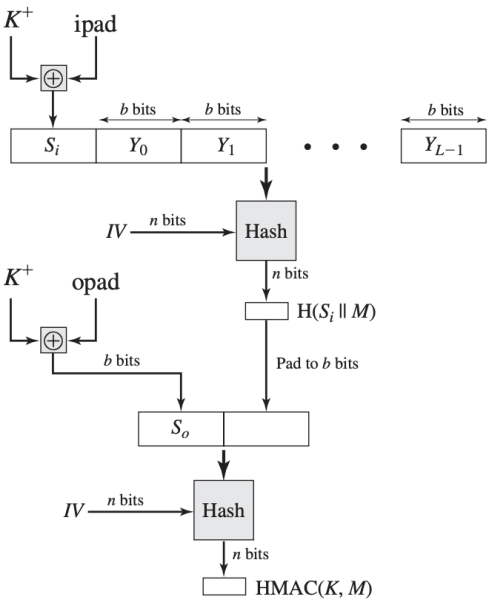
- H = embedded hash function (e.g., MD5, SHA-1, RIPEMD-160)
- IV = initial value input to hash function
- M = message input to HMAC (including the padding specified in the embedded hash function)
- Y_i = i^{th} block of M, $0 <= i <= (L - 1)$
- L = number of blocks in M
- b = number of bits in a block
- n = length of hash code produced by embedded hash function
- K = secret key; recommended length is >= n; if key length is greater than b, the key is input to the hash function to produce an n-bit key

HMAC (RFC 2104)

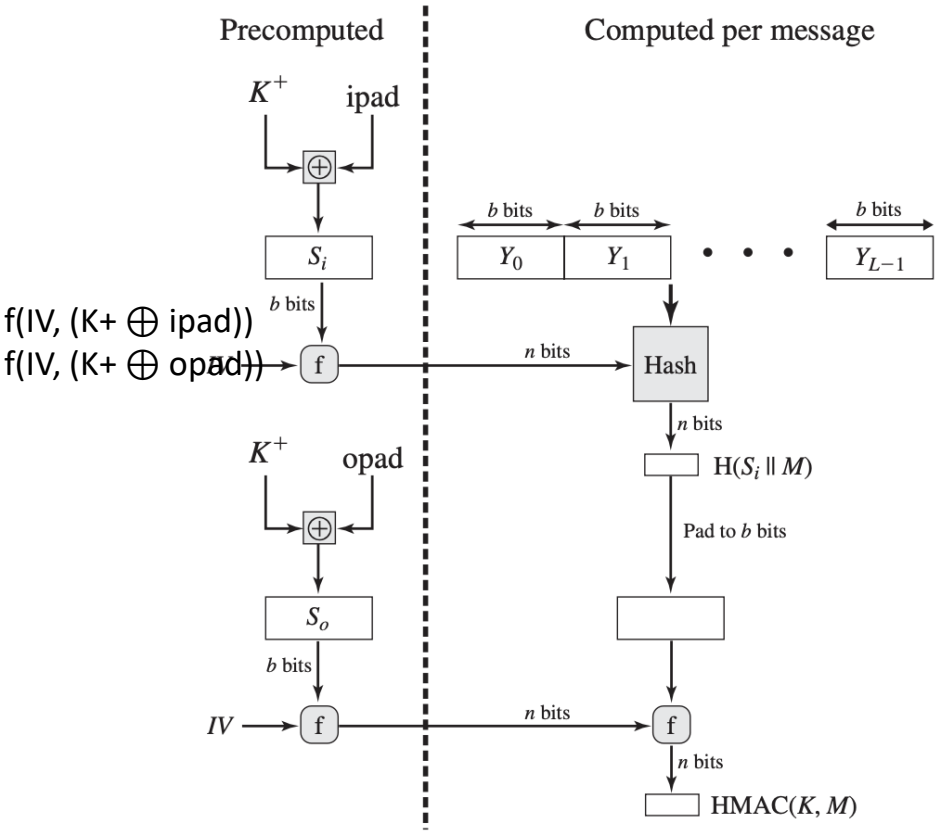
- K^+ = K padded with zeros on the left so that the result is b bits in length
- ipad = 00110110 (36 in hexadecimal) repeated $b/8$ times
- opad = 01011100 (5C in hexadecimal) repeated $b/8$ times
- Then HMAC can be expressed as
  $\mathrm{HMAC}(K, M) = \mathrm{H}[(K^+ \oplus \mathrm{opad}) \,||\, \mathrm{H}[(K^+ \oplus \mathrm{ipad}) \,||\, M]]$

HMAC (RFC 2104)

1. Append zeros to the left end of *K* to create a *b*-bit string *K*+ (e.g., if *K* is of length 160 bits and *b* = 512, then *K* will be appended with 44 zeroes).
2. XOR (bitwise exclusive-OR) K+ with ipad to produce the b-bit block S_i.
3. Append M to S_i.
4. Apply H to the stream generated in step 3.
5. XOR K+ with opad to produce the b-bit block So.
6. Append the hash result from step 4 to So.
7. Apply H to the stream generated in step 6 and output the result.

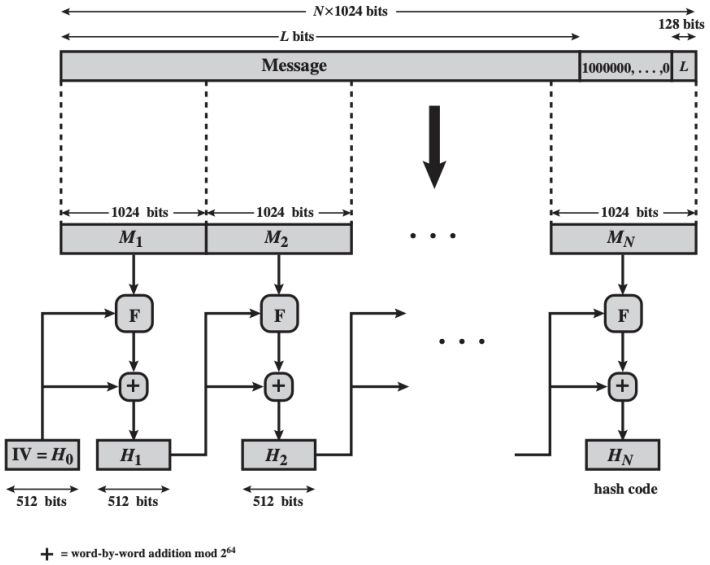
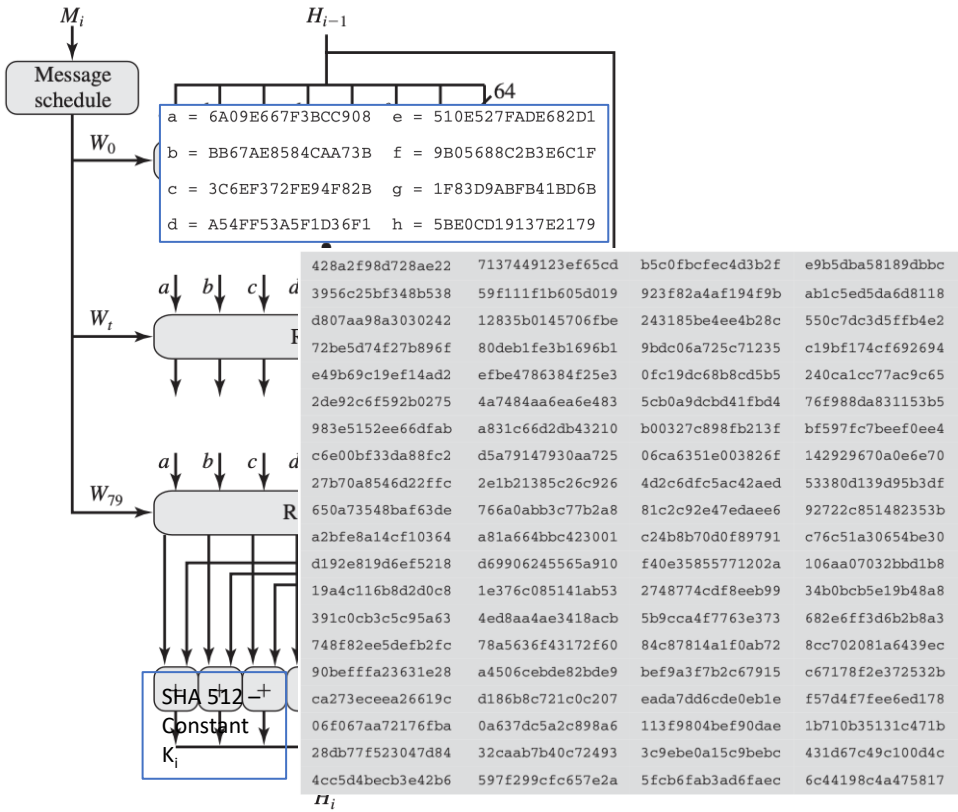


Security of HMAC



Secure Hash Algorithm (SHA)

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80



Secure Hash Algorithm – SHA 512